Development environments

Software Management & Development · Session 1



IE University · MFT & MCSDI

Course overview

- Build one application end to end: Python backend, database, REST API, web frontend
- Test it, package it in containers and deploy it to Azure
- Work like a software team: Git, code review, small deliveries
- Every session: short concepts, then a lab. A group project follows the same path

The running example: ledger

A personal-finance app: accounts, transactions, balances and spending by category. It grows every session:

Sessions	What ledger becomes
S3–S6	a Python project on your laptop
S7–S10	+ PostgreSQL, an API, tests and a web frontend
S11	all of it running in Docker Compose
S12–S13	the same images running on Azure

Why environments matter

Why code breaks on another machine

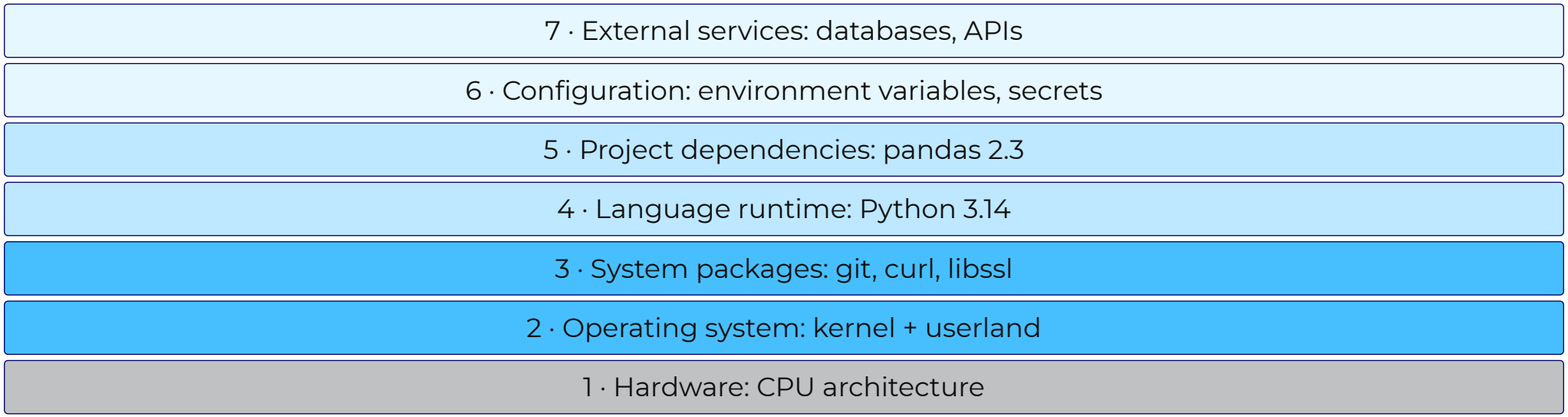
Ana's fraud-detection script works on her laptop. On the server:

1. `ModuleNotFoundError: No module named 'pandas'` — *dependencies*
2. `SyntaxError` on a `match` statement (Python 3.9 vs 3.12) — *runtime*
3. `C:\Users\ana\Desktop\cards.csv` not found on Linux — *operating system*
4. Runs fine… but flags almost nothing: `RISK_THRESHOLD` was set only on Ana's laptop — *configuration*

> None of these is a bug in Ana's logic. The last one is the most dangerous, because it seems to work.

The layers of an environment

Everything except your code that decides how it runs. A difference in any layer can break it.



> The goal: the same environment everywhere (every laptop, the test server, production), or at least a known one.

Reproducibility: three levels

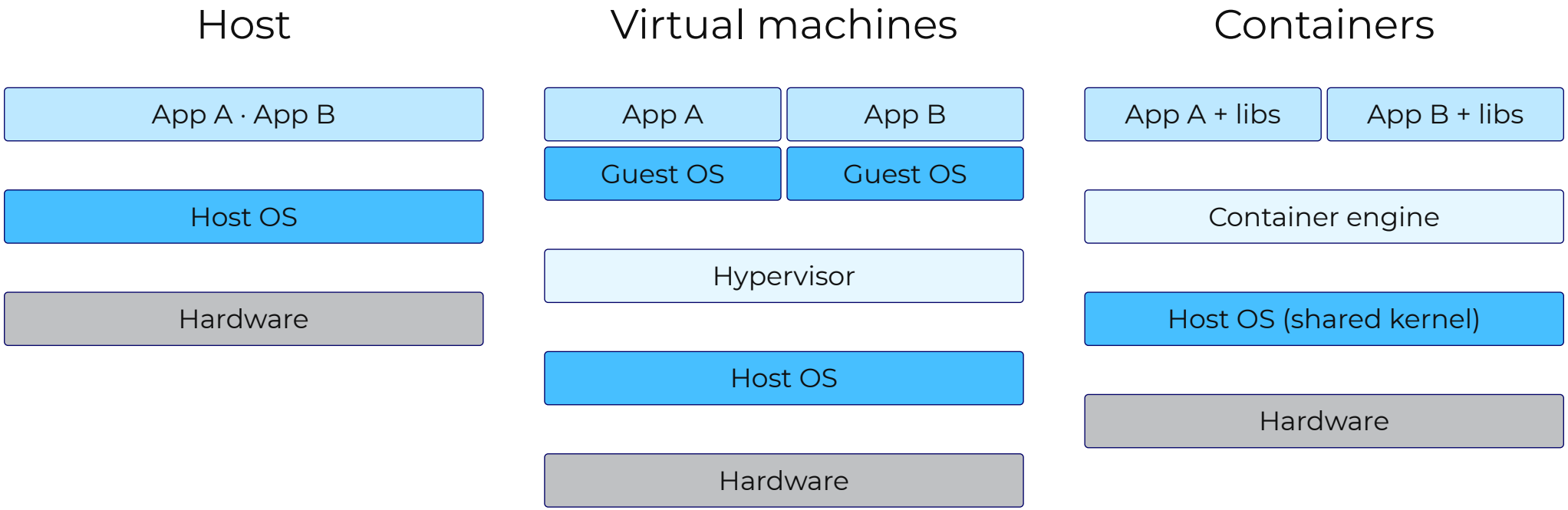
An environment is reproducible if anyone can rebuild it from files and get the same result.

Level	Example	What it fixes
1. Document	a README: “install Python 3.14, then…”	little: people skip steps and it goes out of date
2. Automate	`pyproject.toml` + `uv.lock`, `.env.example`	the runtime and the dependencies
3. Package	a VM image or a container image	OS, runtime and dependencies, shipped with the code

> If it isn’t in a file in the repository, it isn’t reproducible.

Host, virtual machine, container

Three ways to run software



- Host: everything is shared, so upgrading something for one project can break another
- Virtual machine: a full computer in software, with its own kernel
- Container: isolated processes that share the host's kernel

Virtual machines and containers compared

	Virtual machine	Container
Isolates	the whole computer	processes, files, network
Own kernel	yes	no: shares the host's
Starts in	tens of seconds	under a second
Size	gigabytes	megabytes to hundreds of MB
Different OS?	yes (Linux on a Mac…)	same kernel type only
Use it for	a full separate machine	packaging and shipping one app

> Docker Desktop on macOS and Windows runs a small Linux VM in the background. Your containers live inside it.

Processor architectures

- `x86_64` = `amd64`: most Windows PCs and cloud servers
- `arm64` = `aarch64`: Apple Silicon Macs, Snapdragon PCs, some cloud servers

Compiled code runs only on its own architecture. Python is portable, but NumPy, database drivers and cryptography libraries contain compiled parts.

The classic cloud failure

Build an image on an M-series Mac, push it to an `x86_64` server, and it fails with `exec format error`. The fix is to build for the target platform (`linux/amd64`), which we do in session 13.

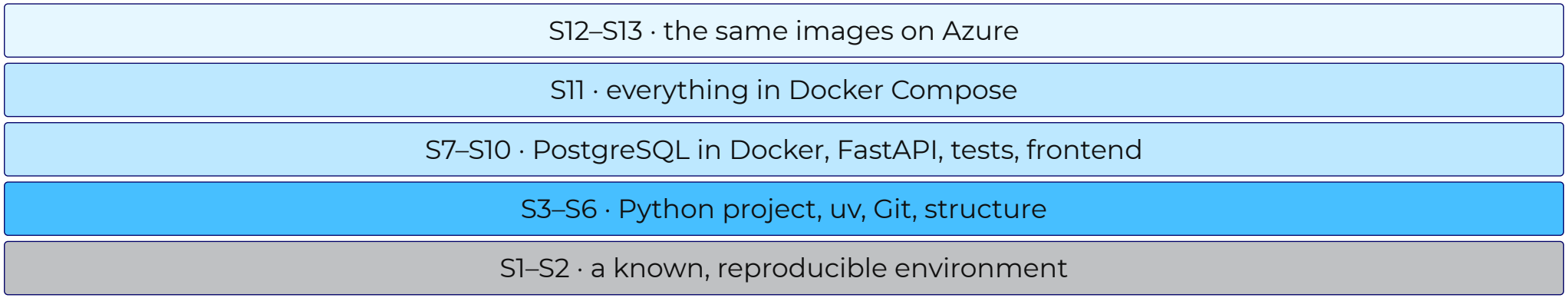
From your laptop to the cloud

Cloud service models: who manages what

Model	You manage	The provider manages	Azure example
On-premises	everything	nothing	—
IaaS	OS, runtime, deps, app, data	hardware, network, virtualization	Virtual Machines
Serverless containers	your image + its configuration	everything below + scaling	Container Apps
Managed services	your data and settings	the whole service	PostgreSQL, Static Web Apps
SaaS	how you use it	everything	Microsoft 365, GitHub

> The container image is yours in every model, and it is the unit that travels unchanged from laptop to cloud.

Course map: from laptop to Azure



The syllabus uses Google Cloud names. We use their Azure equivalents: Artifact Registry → Container Registry, Cloud Run → Container Apps, App Engine → Static Web Apps.

Lab 1: know your machine

Lab 1 steps

1. Open a terminal: `whoami`, `pwd`, `ls`
2. Operating system and architecture: `uname -m`, `sw_vers`…
3. Memory, disk and virtualization
4. Can you install software?
5. Create your GitHub account
6. Fill in your machine card and submit it on Blackboard

> Nothing gets installed today: every command only reads information. In session 2 we use your machine card to set up the course environment.